

AWS EKS 기반 CNAPP 프로젝트

팀 R&R 역할 체크

프로젝트 목표

EKS 환경에서 발생할 수 있는 컨테이너 이미지 취약점, 잘못된 보안 설정, 과도한 IAM·RBAC 권한, 외부 노출, 운영 중 위협을 배포 전부터 운영 단계까지 지속적으로 점검한다.

프로젝트 초반에는 AWS 인프라와 EKS 구성 요소를 정확히 이해하기 위해 **AWS Console에서 EKS 환경을 직접 구축**한다. 이후 동일한 환경을 Terraform으로 다시 구성하여 IaC 기반 자동화와 보안 검증 환경으로 확장한다.

위험한 설정이나 취약점은 배포 전에 차단하고, 배포 후에는 공격 시나리오와 로그를 통해 탐지 가능 여부를 확인한다. 발견된 문제는 설정과 정책에 다시 반영하여 **예방 → 점검 → 탐지 → 개선 → 재검증**으로 이어지는 보안 관리 체계를 구축한다.

1. 역할 간 협업 흐름

AWS Console로 기본 인프라 및 EKS 환경 구축

- AWS·EKS 구성 요소와 연결 관계 이해
- Terraform으로 동일 환경 IaC화
- 애플리케이션 및 보안 테스트 환경 구성
- 배포 전 코드·이미지 보안 점검
- 네트워크 노출 및 EKS 설정·권한 점검
- 공격 시나리오를 통한 위험 검증
- 운영 중 로그 및 위협 탐지
- 문제 개선 후 CI/CD에서 재검증

외부 노출 → Pod → IAM 권한 → AWS 리소스 접근처럼 실제 위협으로 이어질 수 있는 대표적인 경로를 공격 시나리오로 구성하여 보안 설정의 위험성을 검증한다.

2. 착수 계획 — 1~2주차

초기에는 Terraform 자동화보다 **AWS 인프라와 EKS 구조를 직접 이해하는 것을 우선한다.**

먼저 AWS Console에서 네트워크와 EKS 환경을 직접 구축하면서 각 AWS 리소스가 어떤 역할을 하고 서로 어떻게 연결되는지 확인한다. 이후 같은 환경을 Terraform으로 다시 구성하여 수동 구축과 IaC 구축의 차이를 비교하고, 이후 보안 검증에 사용할 공통 환경으로 만든다.

1단계 — AWS Console 기반 EKS 환경 구축

팀 공통

- VPC 구성 확인 및 생성

- Public·Private Subnet 구성
- Internet Gateway·Route Table 구성 확인
- Security Group 구성
- EKS Cluster 생성
- Worker Node 또는 Node Group 생성
- EKS Cluster와 Node 간 연결 확인
- kubectl을 이용한 Cluster 접속 확인
- 테스트용 Pod 및 Service 배포
- Ingress 또는 LoadBalancer를 통한 외부 접근 확인

학습 목표

- VPC·Subnet·Route Table 관계 이해
- EKS Control Plane과 Worker Node 역할 이해
- Security Group이 실제 통신에 미치는 영향 확인
- IAM Role이 EKS Cluster와 Node에 어떻게 적용되는지 확인
- Pod·Service·Ingress 간 트래픽 흐름 이해

2단계 — Terraform으로 EKS 환경 재구축

Console에서 구축한 환경을 기준으로 동일하거나 유사한 환경을 Terraform으로 다시 만든다.

팀 공통

- Terraform 기본 구조 작성
- VPC·Subnet IaC 구성
- Security Group IaC 구성
- EKS Cluster IaC 구성
- Node Group IaC 구성
- IAM Role 및 Policy IaC 구성
- Terraform Plan·Apply 결과 확인
- Console 구축 환경과 Terraform 구축 환경 비교
- Terraform 코드 변경에 따른 인프라 변경 과정 확인

학습 목표

- 수동 구축과 IaC 방식의 차이 이해
- AWS 리소스 간 의존 관계 이해
- Terraform State와 리소스 관리 방식 이해
- 잘못된 설정이 코드에 포함될 경우 반복 배포될 수 있다는 IaC 보안 위험 이해

3단계 — 보안 검증 환경 구성

Terraform 기반 EKS 환경이 완성되면 팀별 역할에 따라 보안 검증을 시작한다.

네트워크 보안

- VPC·Subnet·Security Group 구성 점검

- Ingress·LoadBalancer 등 외부 노출 확인
- 불필요하게 열린 포트 및 접근 경로 확인
- Kubernetes NetworkPolicy 적용
- VPC Flow Logs 및 ALB 로그 수집 준비

클라우드 보안

- Terraform 보안 설정 점검
- Dockerfile 및 컨테이너 이미지 보안 스캔
- Kubernetes 보안 설정 점검
- IAM·RBAC·IRSA 권한 점검
- Critical·High 위험 발생 시 CI/CD 배포 차단 기준 정의

보안엔지니어

- CloudTrail 연동
- GuardDuty 연동 및 탐지 확인
- SentinelOne Runtime 탐지 환경 구성
- Kubernetes Audit Log 수집
- Grafana 기본 모니터링 구성
- 공격 테스트 시 확인할 로그 항목 정리

3. 프로젝트 최소 완료 기준

프로젝트의 핵심 기능을 먼저 완성하는 것을 우선한다.

- AWS Console 기반 EKS 환경 구축 및 구조 이해
- Terraform 기반 EKS 환경 재구축
- EKS 및 AWS 보안 설정 점검 항목 **8~10건 이상**
- IAM·RBAC·IRSA 권한 위험 점검 항목 **4~6건 이상**
- Terraform 및 컨테이너 이미지 보안 스캔 구성
- Critical·High 위험 발생 시 **CI/CD 배포 자동 차단 1건 이상**
- 실제 위험을 확인하기 위한 **통제된 공격 시나리오 2건 이상**
- CloudTrail·SentinelOne·Kubernetes Audit Log 등을 통한 탐지 결과 확인
- 취약 설정 개선 전·후 결과 비교

시간적 여유가 있을 경우 Grafana 대시보드 고도화, MITRE ATT&CK·OWASP Kubernetes Top 10 매핑, 위험도 점수화 등을 추가한다.

4. 네트워크 보안 엔지니어

역할

EKS와 AWS 네트워크에서 외부에 노출된 지점과 실제 접근 가능한 경로를 점검한다.

초기 Console 구축 단계부터 VPC·Subnet·Security Group 구조를 직접 확인하고, 외부 사용자가 어떤 경로로 EKS 서비스에 접근하는지 이해한다.

이후 Terraform으로 동일한 네트워크 환경을 구성하고, 불필요하게 열린 네트워크 설정을 찾아 개선한다.

주요 업무

- AWS VPC·Subnet 구조 이해 및 구축
- Route Table·Internet Gateway 구성 확인
- Security Group·NACL 설정 점검
- Kubernetes NetworkPolicy 구성 및 점검
- Ingress·LoadBalancer 외부 노출 서비스 확인
- 외부에서 서비스 및 Pod까지 접근 가능한지 테스트
- VPC Flow Logs·ALB 로그 확인
- 불필요한 외부 노출 및 허용 규칙 개선
- 개선 전·후 네트워크 상태 비교

핵심 산출물

- AWS·EKS 네트워크 구성도
- 외부 노출 서비스 목록
- Security Group·NACL·NetworkPolicy 점검 결과
- 불필요한 외부 접근 개선 결과
- 네트워크 접근 테스트 및 로그 자료

핵심 질문

“사용자는 어떤 네트워크 경로를 통해 EKS 서비스에 접근하며, 불필요하게 노출된 부분은 없는가?”

5. 클라우드 보안 엔지니어

역할

AWS·EKS 환경의 보안 설정과 권한을 점검하고, 배포 전에 위험한 설정과 취약점을 발견해 차단한다.

Console 구축 과정에서는 IAM Role, EKS 권한, Security Group 등 주요 AWS 설정이 실제로 어떻게 연결되는지 이해한다.

이후 이를 Terraform 코드로 전환하여 보안 설정을 코드 수준에서 점검하고, Dockerfile·컨테이너 이미지·Kubernetes 설정·IAM·RBAC·IRSA 권한까지 배포 전 검증 체계를 구축한다.

주요 업무

- AWS Console에서 EKS·IAM 구성 이해
- EKS Cluster Role·Node Role 확인
- Terraform IaC 보안 설정 점검
- Dockerfile 보안 설정 점검
- 컨테이너 이미지 취약점 스캔
- Kubernetes Pod 및 클러스터 보안 설정 점검
- IAM Role 권한 점검

- Kubernetes RBAC 과다 권한 점검
- IRSA와 IAM Role 권한 관계 확인
- Critical·High 위험 발견 시 CI/CD 배포 자동 차단
- 발견된 문제 수정 후 다시 스캔하여 개선 여부 확인

필요한 경우 외부에 노출된 Pod가 IRSA 또는 Node IAM Role 등을 통해 AWS 리소스까지 접근할 수 있는지 통제된 공격 시나리오로 검증한다.

핵심 산출물

- Console 기반 AWS·EKS 구성 확인 자료
- Terraform 보안 점검 결과
- Dockerfile·컨테이너 이미지 스캔 결과
- Kubernetes 설정 점검 결과
- IAM·RBAC·IRSA 권한 점검 결과
- CI/CD 배포 차단 결과
- 설정 개선 전·후 비교 결과
- 공격 시나리오 검증 결과

핵심 질문

“AWS와 EKS의 권한 및 설정이 어떻게 연결되어 있으며, 위험한 설정을 배포 전에 발견하고 차단할 수 있는가?”

6. 보안엔지니어

역할

배포된 EKS 환경에서 실제 공격 행동과 이상 행위가 발생했을 때 이를 로그와 보안 솔루션에서 탐지할 수 있는지 검증한다.

Console 구축 단계에서 AWS 로그와 보안 서비스가 어떤 리소스의 이벤트를 기록하는지 먼저 이해한다.

이후 CloudTrail, GuardDuty, SentinelOne, Kubernetes Audit Log 등을 이용하여 어떤 공격 행동이 어떤 로그와 탐지 이벤트로 남는지 확인한다.

주요 업무

- CloudTrail 로그 수집 및 분석
- GuardDuty 탐지 이벤트 확인
- SentinelOne을 이용한 Pod·Node Runtime 위협 탐지
- Kubernetes Audit Log 수집 및 분석
- 통제된 공격 시나리오 수행 시 로그 증거 수집
- 공격 행동별 탐지 여부 정리
- Grafana를 이용한 주요 이벤트 모니터링
- 탐지된 위험에 대한 대응 절차 정리

핵심 산출물

- CloudTrail·GuardDuty 탐지 결과
- SentinelOne Runtime 탐지 결과
- Kubernetes Audit Log 분석 결과
- 공격 시나리오별 탐지 결과표
- Grafana 모니터링 화면
- 보안 이벤트 발생 시 대응 절차

핵심 질문

“EKS에서 공격이나 이상 행동이 발생했을 때 어떤 로그가 남고, 이를 실제로 탐지할 수 있는가?”

7. 확장 순서

핵심 기능을 완성한 뒤 다음 순서로 범위를 확장한다.

1단계. AWS Console 기반 EKS 환경 구축

직접 AWS 리소스를 생성하면서 VPC·Subnet·Security Group·IAM·EKS 구조와 각 리소스 간 연결 관계를 이해한다.

2단계. Terraform IaC 전환

Console에서 구축한 환경을 Terraform으로 다시 구성하여 반복 가능한 EKS 배포 환경을 만든다.

3단계. 배포 전 보안 점검

Terraform·Dockerfile·컨테이너 이미지 스캔과 CI/CD 배포 차단 기능을 구성한다.

4단계. EKS 설정 및 권한 점검

Kubernetes 설정, IAM·RBAC·IRSA 과다 권한을 점검하고 개선한다.

5단계. 공격 시나리오 검증

외부 노출 → Pod → AWS 권한처럼 실제 위협으로 이어질 수 있는 대표적인 시나리오를 통제된 환경에서 검증한다.

6단계. Runtime 탐지

CloudTrail·GuardDuty·SentinelOne·Kubernetes Audit Log에서 공격 행동이 탐지되는지 확인한다.

7단계. 모니터링 및 분석 고도화

Grafana Dashboard, MITRE ATT&CK·OWASP Kubernetes Top 10 매핑, 위험도 점수화 등을 추가한다.